

PARTIU MARROCOS / KEROMINDPARTIU MARROCOS / KEROMIND

Payment Foundation 01 — Relatório de Fechamento

Relatório de Fechamento

15 de setembro de 2026

Etapa 2 de 8 — PM-NIGHT-RUN-02 (COMANDO DE EXECUÇÃO CONTINUADA CONTROLADA)

Executada sob autorização já concedida ("AUTORIZADA SE BOOKING = GREEN").

Objetivo

Construir um domínio de pagamento **provider-neutral** (§10 do comando: "PAYMENT ≠ GATEWAY") — pagamento único, sinal+saldo ou parcelamento, sem acoplar a nenhum gateway específico (Stripe/Mercado Pago/PIX), preparado para webhook futuro, com estorno sempre passando por aprovação humana via Gate.

PAYMENT DOMAIN IMPLEMENTED — não PAYMENT PROVIDER LIVE

Distinção exigida explicitamente pelo comando (§15), afirmada aqui sem ambiguidade: **nenhum gateway real foi integrado nesta rodada**. Toda operação é modo **MANUAL/OFFLINE** (**provider: null**) — um humano registra que recebeu um pagamento (ex.: PIX confirmado no extrato, dinheiro em mãos), o sistema nunca chama nenhuma API de pagamento. Nenhuma credencial de gateway existe, foi solicitada ou foi inventada.

Arquitetura

Provider-neutral de verdade, não só de nome

Payment.provider/**Payment.providerReference** são campos opcionais opacos — hoje sempre **null**. **Payment.method** é texto livre (ex.: "pix", "cartão") só para descrição, nunca usado em lógica de negócio (nenhum **if method === "pix"** em nenhum lugar do código). Quando um gateway real for integrado no futuro, ele preenche esses campos — nenhuma migration ou mudança estrutural é necessária para isso.

Parcelamento sem campo extra

Um Booking pode ter N **Payment**. Pagamento único é um Booking com um Payment; sinal+saldo ou parcelamento são vários Payment com **vencimento** diferentes. Nenhum campo de "plano de parcelamento" foi necessário — a própria tabela já suporta os três casos citados pelo comando (§12) sem estrutura adicional.

Sincronização Booking ↔ Payment, sem regressão

sincronizarStatusPagamentoBooking soma os **Payment** com status **PAGO** de um Booking e decide **AGUARDANDO_PAGAMENTO/PAGAMENTO_PARCIAL/PAGO** — mas **nunca mexe** num Booking que já avançou além desses dois estados iniciais (ex.: já **CONFIRMADA/EM_OPERACAO**) e **nunca força** uma transição que a máquina de estados do Booking (Etapa 1) não permita. Testado explicitamente: registrar um pagamento extra num Booking já **CONFIRMADA** não regride o status.

Estorno SEMPRE via Gate FINANCEIRO — nunca autoaprovado (§13/§62)

solicitarEstornoPayment só aceita a solicitação a partir de **PAGO/PARCIALMENTE_PAGO**, valida o valor (>0 e $\leq$ valor pago), e cria um **Gate** categoria **FINANCEIRO** de verdade (T1) — o pagamento fica com **gateId** setado, mas **nada muda até aprovação humana real**.

confirmarEstornoAposAprovacaoGate só aplica o estorno depois de **decidirGate** (decisor sempre um **User** membro do tenant, nunca "yalla"/"sistema").

Idempotência real, não só nominal: diferente do padrão de Proposal (onde reaplicar "ENVIADA" é inofensivo), reaplicar um estorno cegamente DUPLICARIA o valor devolvido.

confirmarEstornoAposAprovacaoGate detecta que o Payment já não está mais em **PAGO/PARCIALMENTE_PAGO** (ou seja, o Gate já foi consumido numa chamada anterior) e retorna o

estado já aplicado (`jaAplicado: true`) em vez de estornar de novo — testado explicitamente: confirmar duas vezes o mesmo Gate aprovado nunca duplica `valorEstornado`.

Idempotência preparada para webhook futuro (§14)

`idempotencyKey` (único por tenant, nulo permitido em múltiplas linhas) já existe no schema e é respeitado por `criarPayment` — chamar duas vezes com a mesma chave nunca cria um segundo Payment. Nenhum endpoint de webhook foi construído (não há gateway pra emitir webhook ainda) — só o contrato de deduplicação, pronto pra quando existir.

"Vencimento" é informação, não estado

O comando lista

`PENDING/PROCESSING/PAID/PARTIALLY_PAID/FAILED/CANCELLED/REFUNDED/PARTIALLY_REFUNDED` como estados de Payment — não há um estado "vencido" nessa lista. `vencimento` é um campo informativo (comparável com `now()` em qualquer consulta/UI) — não inventamos um estado extra fora do que foi pedido.

Migrations

1 nova, puramente aditiva (1 tabela nova, nenhuma alteração em tabela existente):

```
20260915050000_payment_foundation_01      -- enum PaymentStatus + tabela payments
20260915050100_enable_rls_payment_foundation_01  -- RLS
```

35 migrations no total (era 33 ao final da Etapa 1).

RBAC

3 permissões novas: `payments.view`, `payments.manage` (criar cobrança, registrar resultado manual), `payments.refund` (solicitar estorno — separada de `payments.manage` de propósito, mesma lógica de `gates.decide`: mover dinheiro de volta é decisão de maior risco). Administrador ganha as três; Vendas ganha `view+manage` mas não `refund`; Atendimento ganha só `view`.

Segurança

- RLS testado (Tenant B não lista Payment de A; tentar criar Payment em Booking de outro tenant falha porque o Booking nem é visível sob RLS).
- `PAYMENT_CRIADO/PAYMENT_REGISTRADO/PAYMENT_CONFIRMADO/PAYMENT_FALHOU/PAYMENT_CANCELADO/PAYMENT_ESTORNADO` auditados (T1).
- Nenhuma credencial de gateway em código/log/documento — não existe nenhuma nesta rodada.

Testes novos (34)

- `packages/db/tests/unit/payment-transicoes.test.ts` — 8 (máquina de estados pura: caminho feliz, retry após falha, parcial evolui pra pago/estornado, pago só estorna nunca cancela, sem pular de PENDENTE pra REEMBOLSADO, terminal não sai do lugar, sem autotransição).
- `packages/db/tests/integration/payment.test.ts` — 13 (criação/idempotência, sincronização com Booking single/parcelado, transição inválida rejeitada, nunca regride Booking avançado, estorno só a partir de PAGO/PARCIAL, valor inválido rejeitado, Gate real criado e só aplica após aprovação, Gate rejeitado nunca aplica, **dupla confirmação nunca duplica valor estornado**, resumo de pagamento, isolamento multi-tenant).

Regressão

399/399 testes passando (378 ao final da Etapa 1 + 21 novos: 8 unitários + 13 de integração). Typecheck limpo. Build limpo — 17 rotas (`/leads/[id]` cresceu de 4.38kB pra 5.33kB — painel de pagamentos).

Verificação ponta a ponta (navegador real)

Fluxo completo exercido de verdade: proposta R\$ 12.000 → aceita → reserva criada → primeira parcela de R\$ 4.000 criada (**PENDENTE**) → marcada como paga → **Booking sincroniza sozinho pra "Pagamento parcial"** (4.000 < 12.000, transição correta) → "Solicitar estorno" de R\$ 1.000 com motivo → Gate real aparece em **/gates**, categoria "Financeiro", com o valor e motivo exatos → aprovado → "Verificar estorno" → pagamento vira **"Parcialmente reembolsado"** de verdade. Zero erros de console em todo o fluxo, incluindo o caminho de Gate. Dados de verificação removidos/restaurados ao final.

Limitações

Nenhum gateway real integrado (deliberado, fora de escopo sem credencial/autorização — §15); **method** é texto livre, sem taxonomia fechada; nenhum job de "vencimento próximo"/lembrete automático ainda (isso é natural para Notifications Foundation, Etapa 7); reembolso total (não parcial) em múltiplas etapas soma corretamente mas não há teste específico de "3 estornos parciais sucessivos" além do padrão de 1-estorno-completo/1-parcial já cobertos.

Decisões do fundador pendentes

Nenhuma nova. Qual gateway real integrar (se algum) permanece uma decisão de negócio futura, fora desta fundação.

Resultado / Status

GREEN — domínio provider-neutral implementado (nunca "provider live"), parcelamento suportado sem campo extra, sincronização Booking↔Payment sem regressão, estorno sempre via Gate real com idempotência genuína (não duplica), idempotência de submissão preparada para webhook futuro, 34/34 testes novos passando, regressão completa (399/399), typecheck/build limpos, migration única aditiva com RLS, RBAC estendido, verificação ponta a ponta completa em navegador real incluindo o caminho de Gate.

Próximo bloco

Etapa 3 — Finance Core Real 01. **Prosseguindo automaticamente conforme autorização.**